Pluribus:
Predictive Distributed Caching for LLM Coding Agents

Devon Tietjen (@d-tietjen)

July 6, 2026

Contents

Abstract

Pluribus is a distributed cache optimizer for private LLM serving clusters. It keeps values in local shardmap and shardcache stores, publishes compact ownership sketches through Blossom application-state blocks [6], and pulls remote values only when local demand makes reuse likely. This paper describes the Pluribus protocol, its metadata-blind safety boundary, and the optional semantic path used when applications can supply embeddings or value classes, following the same vector-space assumption used by semantic caching and dense retrieval systems [8, 10, 12]. We focus on a shared coding repository workload where thousands of LLM instances operate on the same project. In that setting, query and cache embeddings cluster on a common project manifold, making semantic cache reuse far stronger than in a many-repository workload. A deterministic 64-cache-node benchmark with 4,096 logical LLM instances reaches 100.00% subsystem-level hit rate and 100.00% top-1 subsystem prediction in the shared-repo case. In the Figure 6 coverage grid, the same cache capacity over one 8x larger monorepo surface reaches 83.24% subsystem hit rate, a 32x monorepo reaches 64.55%, and a normalized 128-repository breadth case reaches 25.63% once finite sketch visibility and metadata saturation dominate. The formerly weak 16-repository breadth point now reaches 96.03% after occupancy-aware range merging. The result supports Pluribus as a cache layer for coding agents, repository question answering, code review, and repeated refactor planning over shared codebases, while showing that repository working-set size must be modeled alongside repository count.

1 Introduction

LLM coding workloads repeatedly ask similar questions about the same codebase: which files define an interface, which tests cover a module, where a type is constructed, or how a recent failure relates to a patch. When thousands of LLM instances serve a shared project, their prompts and retrieval artifacts differ in detail but remain semantically close. A cache that treats each instance as isolated throws away this shared locality, echoing the broader observation that LLM-serving systems lose efficiency when reusable KV and context state are managed independently per engine or request [3, 4].

Pluribus is designed to recover that locality without centralizing the cache. Each node keeps its own embedded store. Blossom carries compact cache coverage metadata, not cache values [6]. A node that observes demand compares that demand with peer sketches, requests only bounded candidate ranges, validates the response, and inserts useful values into its local store. When embeddings are available, this behaves like a distributed semantic cache [7, 8]. When embeddings are not available, Pluribus falls back to blind operational learning over opaque keys, byte sizes, TTLs, and reuse outcomes.

This paper follows the same high-level organization as the Blossom paper [6]: terminology, related work, assumptions, protocol, projections, benchmarks, future additions, and conclusion. The protocol is narrower than Blossom

consensus. Pluribus does not attempt global agreement over cache contents or Byzantine consensus [25]. Its goal is local performance improvement under bounded metadata, explicit data movement, and conservative validation.

The main contribution of this revision is the shared-repository benchmark. It models 4,096 logical LLM inference instances working against one coding repository over a 64-node cache cluster. The benchmark intentionally makes in-project embeddings close while preserving subsystem and file offsets, so the test can distinguish broad project reuse, subsystem reuse, and exact-file reuse. The result is the expected best case for Pluribus: every query is served locally or repaired through a small remote pull at the subsystem level.

2 Terminology

Cache node
A Pluribus process with an embedded shardmap store, shardcache access, a local sidecar index, and a local planner.

Logical LLM instance
A worker running inference. Many logical instances may share one physical cache node, following the common separation between logical requests and physical serving engines in high-throughput LLM serving [2, 3].

Application state
A bounded Blossom payload containing Pluribus cache coverage sketches [6]. It advertises where useful values may exist; it does not contain full values.

Sidecar
Metadata Pluribus stores beside a cache value: key hash, projected embedding, model id, value class, TTL, byte count, hotness, and validation fields.

Demand signal
A local observation that a node may need nearby cache values. In metadata mode this includes an embedding. In blind mode it is derived from opaque operational events.

Range sketch
A compact summary of nearby cached values owned by one peer. It includes a centroid, coarse radius, ellipsoid shape, value class, count, bytes, hotness, and freshness bounds. The centroid/range view follows standard vector-space retrieval practice [11, 12].

Semantic hit
A cache hit where the returned value is close enough to the query embedding and passes application policy [7, 9]. This is distinct from an exact-key hit.

Subsystem hit
In the shared-repo benchmark, a semantic hit whose key belongs to the same repository subsystem as the query.

3 Related Work

KV-cache management is now a first-order serving problem for transformer inference [1]. PagedAttention and vLLM showed that block-level KV-cache management can reduce memory waste and enable cache sharing within an inference engine [2]. LMCache extends this direction by exposing KV caches as a reusable storage and communication layer across LLM engines and queries [3]. Pluribus assumes those lower storage and runtime layers can already move or persist values; its contribution is the predictive metadata policy that decides which remote values are worth pre-seeding.

Mooncake is the closest systems baseline. It uses a KVCache-centric disaggregated architecture with a global cache pool, transfer engine, and cache-aware scheduler to trade more storage for less computation in LLM serving [5]. Pluribus targets a different point in the design space: no central cache owner, no broadcast replication of values, and a local planner that pulls bounded peer ranges from compact ownership sketches carried through Blossom application state [6].

Semantic caches reduce duplicate work by accepting near-neighbor requests rather than exact-key matches [7, 8, 9], but a centralized semantic cache can add a request-path dependency and concentrate memory pressure. Peer-to-peer caches and distributed hash tables preserve local failure domains, but naive broadcast wastes bandwidth and memory [21, 22]. Pluribus occupies the middle ground: values remain local, while compact sketches make remote locality discoverable. Its geometric path uses standard vector-space retrieval machinery: normalized embeddings, cosine similarity, and nearest-neighbor range search [11, 12].

Traditional distributed caches usually optimize exact-key placement. That is necessary for object identity and follows a long line of distributed key-value systems [22], but it misses the common LLM coding case where nearby prompt fragments, summaries, retrieval chunks, or analysis artifacts can be reused even when the exact prompt is not repeated. Retrieval-augmented generation makes these retrieved chunks first-class model context [10]. Pluribus can use exact keys when they are known, but its optional semantic path is designed for near-neighbor reuse.

LLM KVCache systems present a stricter boundary. Many production cache integrations expose opaque keys and byte values, not prompt spans, embeddings, tenant labels, or transformer KV layout. Pluribus therefore keeps a metadata-blind mode as the baseline. The shared-repo path is an accelerator for applications that can safely supply embeddings and governance metadata; it is not required for correctness.

4 Assumptions

Pluribus assumes a private trusted cluster. Blossom provides a bounded metadata transport and node membership context [6], but Pluribus does not require public Byzantine consensus over cache contents [25]. A peer sketch is an advertisement, not a proof that a value must be trusted.

The cache data plane is explicit. Values move only through pull requests and responses. A receiver validates model id, embedding dimension, TTL, duplicate items, and value hash before inserting data locally. A bad response should fail closed and become peer-quality evidence.

The application decides what a semantic hit may be used for. In a shared code repository, a nearby subsystem artifact may be useful prompt context. In a strict object cache, only exact-key matches may be acceptable. Pluribus exposes both modes: semantic range pull and exact-key pull, matching the distinction between semantic caching and exact KV-cache reuse [9, 5].

The benchmark assumptions are intentionally transparent. The shared-repo test uses synthetic embeddings shaped to match the hypothesis that thousands of LLM instances over one project occupy a tight semantic region. It models logical instances, cache nodes, peer latency, and metadata budgets in process. It does not claim to measure network sockets, LLM runtime, eviction pressure, or process isolation, which are separate sources of variance in production LLM serving systems [2, 3].

5 Protocol

At write time, a cache node stores the value in shardmap and writes a Pluribus sidecar. In semantic mode, the sidecar records the supplied embedding, model id, value class, governance bytes, TTL, and validation hashes. In blind mode, the sidecar is derived from visible operational fields and deterministic byte projection.

Periodically, each node builds a range sketch from live sidecars:

$$\Sigma_i = \{(c_R, \rho_R, \mathcal{A}_R, N_R, b_R, h_R, v_R)\}_{R=1}^{m_i},$$

where c_R is a projected centroid, ρ_R is the coarse observed cosine-distance envelope, $\mathcal{A}_R$ is the default low-rank ellipsoid descriptor, N_R is raw entry count, b_R is raw byte count, h_R is hotness, and v_R is the value class. The serialized sketch is bounded by Blossom's 16 KiB application-state ceiling per block [6].

Notation is consistent throughout the paper: i, j index cache nodes, R, L index advertised ranges, x indexes cache entries, C_i is node i's local cache contents, and $\mathcal{C}_i$ is reserved for matrices of visible group centroids. Raw range counts and bytes are N_R, b_R; capped planner counts and bytes are n_R, B_R. The scalar s is planner slack, while ℓ denotes a vector of source-group similarities.

5.1 Range Thresholds

Let $z_x = \pi(e_x)$ be the normalized projected embedding for cache entry x, and let $d(a, b) = 1 - a^\top b$ be cosine distance [11]. During sketch publication, Pluribus builds value-class-local ranges with a hotness-weighted centroid:

$$c_R = \operatorname{norm}\left(\frac{\sum_{x\in R} w_x z_x}{\sum_{x\in R} w_x}\right), \qquad w_x = \max(1, h_x).$$

A live sidecar joins the nearest working range only when it is inside the publication threshold:

$$\min_R d(z_x, c_R) \le r_t.$$

Here r_t is `target_r`. Otherwise the entry starts a new range. The range then publishes the p95 member distance as its coarse observed radius:

$$\rho_R = Q_{0.95}\big(\{d(z_x, c_R) : x \in R\}\big).$$

The radius remains the coarse outer envelope. Every range publishes a compact D_e-dimensional ellipsoid descriptor $\mathcal{A}_R = (U_R, a_R, \sigma_{\perp,R}, \tau_R)$. Sparse ranges may publish no retained principal axes, $k_R = 0$, but still carry a residual radius over the full projected space. Let $\Delta_x = z_x - c_R$ and define the hotness-weighted local covariance

$$\Sigma_R = \frac{\sum_{x\in R} w_x \Delta_x \Delta_x^\top}{\sum_{x\in R} w_x}.$$

$U_R = [u_{R1}, \ldots, u_{Rk_R}] \in \mathbb{R}^{D_e \times k_R}$ stores the top orthonormal principal directions of Σ_R, the standard principal component basis for the weighted local covariance [13], with k_R chosen by the metadata budget. The published axis radii and omitted-subspace radius are computed with $P_R^\perp = I - U_R U_R^\top$:

$$a_{R\ell} = Q_{0.95}\big(\{|u_{R\ell}^\top \Delta_x|\}_{x\in R}\big),$$
$$\sigma_{\perp,R} = Q_{0.95}\big(\{\|P_R^\perp \Delta_x\|_2\}_{x\in R}\big).$$

Thus the retained axes explain the dominant range directions, while $\sigma_{\perp,R}$ still charges the full residual energy across every dimension not explicitly advertised.

Figure 2 shows the one-dimensional statistical view behind an advertised axis. Each curve is a weighted empirical distribution projected onto one principal direction: high-reuse entries pull the mean and standard-deviation window more strongly, and the overlap gives the planner an interpretable picture of where local demand and peer ownership are likely to agree before the full n-dimensional gate is evaluated.

For any projected query q, convert cosine-distance slack into Euclidean radius slack with

$$\delta_s = \sqrt{2\max(0, s)},$$

because normalized vectors satisfy $\|a-b\|_2^2 = 2(1-a^\top b) = 2d(a, b)$. Define

$$r_{R\ell}(s) = a_{R\ell} + \delta_s, \qquad r_{\perp,R}(s) = \sigma_{\perp,R} + \delta_s.$$

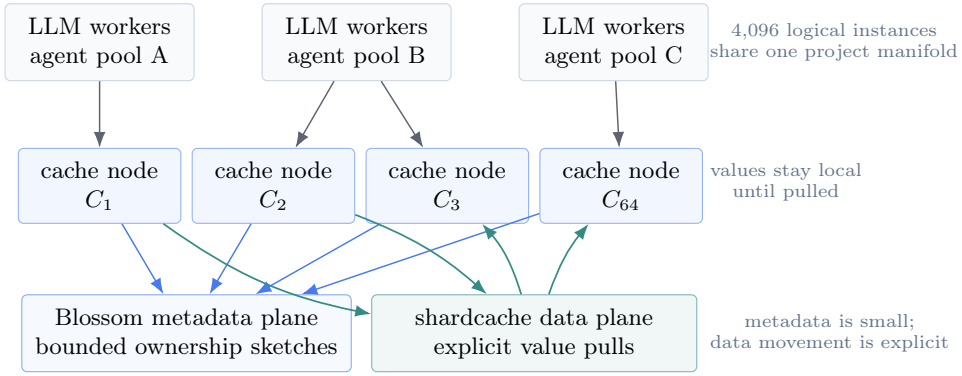

Figure 1: Shared-repository Pluribus topology. Worker pools issue local reads against nearby cache nodes. Blue arrows are Blossom sketch updates: small metadata only. Teal arrows are shardcache pulls: full values move only after a local planner expects reuse.

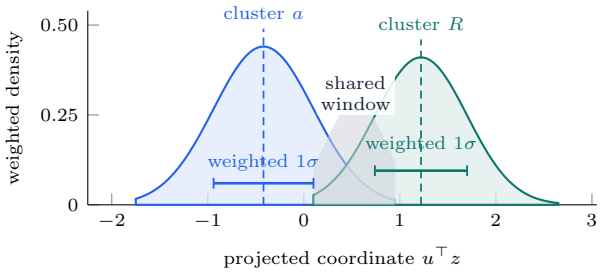

Figure 2: Weighted one-dimensional cluster distributions along a principal range direction. Recent or frequently reused entries contribute more mass to each windowed density; dashed lines mark weighted means, horizontal bars show one weighted standard deviation, and the shaded overlap is the region where a candidate query can be close to both the local group and the peer range before the full n-dimensional ellipsoid score is applied.

The ellipsoid score with planner slack is

$$m_R(q;s)^2 = \sum_{\ell=1}^{k_R} \left(\frac{u_{R\ell}^\top (q - c_R)}{r_{R\ell}(s)} \right)^2 + \left(\frac{\|P_R^\perp (q - c_R)\|_2}{r_{\perp,R}(s)} \right)^2 .$$

Proof sketch. The slack conversion follows from unit-vector geometry. Since both projected vectors are normalized, $\|a - b\|_2^2 = a^\top a + b^\top b - 2a^\top b = 2(1 - a^\top b) = 2d(a, b)$. A cosine-distance slack s is therefore a Euclidean radius slack $\sqrt{2s}$, which is the value added to every published ellipsoid radius in the implementation.

The threshold is fit from members:

$$\tau_R = \max\left(1,\ Q_{0.95}\left(\{m_R(z_x;0)\}_{x\in R}\right)\right).$$

With $k_R = 0$, the descriptor is a residual-only n-dimensional gate; with larger k_R, the shape approaches the full Mahalanobis gate [14] while remaining compact enough for Blossom [6].

The occupancy-aware builder refines mixed ranges before publication. Let $n_\ell = \max(I_{\min}, 4)$ be the leaf floor and $N_{\text{med}} = \max(12, 2n_\ell)$. A range can split only when it has at least $2n_\ell$ members. It is split when

$$\rho_R > r_t \quad \text{or} \quad \sigma_{\perp,R} > r_t \quad \text{or} \quad \tau_R > 1.8,$$

and, on the first refinement pass, a medium-occupancy range $n_\ell < N_R \le N_{\text{med}}$ also splits when

$$\rho_R > 1.5 r_{\min} \quad \text{or} \quad \sigma_{\perp,R} > 1.5 r_{\min} \quad \text{or} \quad \tau_R > 1.15.$$

Underfilled nearby ranges are merged only while the merged p95 radius satisfies $\rho_{R\cup L} \le r_{\max}$. When the range or byte budget is tight, the builder keeps compact hot summaries before broad low-utility summaries, and degrades ellipsoid axes before dropping whole summaries.

Termination and budget proof sketch. The refinement loop has at most four passes, and a split is accepted only when both children contain at least n_ℓ members and the split budget is not exceeded. The underfilled merge loop strictly decreases the number of working ranges on every accepted merge. The final serialization loop either decreases the retained axis count or removes one summary; both quantities are finite. Thus publication terminates. If a single residual-only summary is still larger than the soft target, the builder keeps the summary rather than publishing an empty sketch; the final application-state encoder still enforces the 16 KiB hard cap, so an impossible target fails closed instead of publishing an oversized Blossom payload.

At pull time, node i forms demand $q_i = \pi(e_q)$. It may consider a peer range only if the query first lies inside the coarse advertised radius plus planner slack:

$$d(q_i, c_R) \le \rho_R + s,$$

where s is `max_distance_slack`. The same query must also satisfy $m_R(q_i; s) \le \tau_R$. The geometric confidence

used by the utility model is therefore

$$\gamma_\rho(q_i, R) = 1 - \mathrm{clip}_{[0,1]}\left(\frac{d(q_i, c_R)}{\rho_R + s}\right),$$
$$\gamma_{\mathcal{A}}(q_i, R) = 1 - \mathrm{clip}_{[0,1]}\left(\frac{m_R(q_i; s)}{\tau_R}\right),$$
$$\gamma_s(q_i, R) = \min(\gamma_\rho(q_i, R), \gamma_{\mathcal{A}}(q_i, R)),$$
$$\gamma_r(R) = (1 + \lambda_r \rho_R)^{-1}.$$

Let n_R be entry count capped by $I_{\max}$, B_R be byte count capped by $B_{\max}$, W_{v_R} be the value-class weight, and μ_{v_R} be the value-class byte-cost multiplier:

$$n_R = \min(N_R, I_{\max}), \qquad B_R = \min(b_R, B_{\max}).$$

The current deterministic pull gate includes the sharing-policy gain described in Section 6. Let

$$\Phi_i(q_i, R) = \kappa A_i L_i S_\theta(q_i, R) G_r(q_i, R) \bar{\nu}_i \bar{\mathcal{D}}_i / P_i(R)$$

be the implemented range gain used by the core planner. The planner utility is

$$D_R = 1 + \lambda_d \ln(1 + n_R),$$
$$\Lambda_R^{\text{byte}} = 1 + \lambda_b \mu_{v_R} B_R / B_{\max},$$
$$U_i(q_i, R) = h_{q_i}\, \gamma_s(q_i, R)\, \gamma_r(R)\, \ln(2 + h_R) \cdot D_R W_{v_R} \Phi_i(q_i, R) / \Lambda_R^{\text{byte}}.$$

The planner emits a pull only when $U_i(q_i, R) \geq U_{\min}$, the range is fresh enough, the model and value class match, and raw transformer KV pulls are explicitly allowed [1, 2]. This keeps metadata sharing broad and cheap while making value movement pass a tighter local expected-value gate.

5.2 Predictive Group Vectors

The deterministic gate above checks whether a peer range is near the current demand. The predictive path asks the next question: when demand is near one cache group, which peer-group direction is likely to be useful next? Pluribus answers by making group vectors explicit weighted averages, then moving the query along source-to-peer directions before asking the peer to validate candidate values. A structured cache entry may expose p embedding fields: file path, symbol, prompt fragment, tool result, decoded tensor summary, or another application-provided view. Let $F_x = [e_{x1}, \ldots, e_{xp}] \in \mathbb{R}^{D_e \times p}$ be those field vectors and let $\beta \in \mathbb{R}_+^p$ be field-importance weights. The per-entry vector is

$$z_x = \mathrm{norm}\left(\frac{F_x \beta}{\mathbf{1}^\top \beta}\right).$$

Thus important fields steer the embedding more strongly while still producing a single cosine-ready vector, following the same normalized vector-space assumption used by cosine retrieval [11, 12]. For a range R, stack entry vectors as $Z_R = [z_1, \ldots, z_{n_R}]$ and use access or hotness weights $w_R \in \mathbb{R}_+^{n_R}$:

$$\bar{c}_R = \frac{Z_R w_R}{\mathbf{1}^\top w_R}, \qquad c_R = \mathrm{norm}(\bar{c}_R).$$

Equivalently, if all field tensors are stored as $\mathcal{E}_R \in \mathbb{R}^{D_e \times p \times n_R}$, the same centroid is the mode product

$$\bar{c}_R = \frac{\mathcal{E}_R \times_2 \beta \times_3 w_R}{(\mathbf{1}^\top \beta)(\mathbf{1}^\top w_R)}, \qquad c_R = \mathrm{norm}(\bar{c}_R).$$

The V1 range-sketch code computes the raw average first, then L2-normalizes the advertised centroid so peer sketches stay cosine-ready.

Now let $\mathcal{C}_i = [c_{g_1}, \ldots, c_{g_{J_i}}]$ be the matrix of visible group centroids. In V1 these are advertised peer centroids; the same algebra can also include local group centroids when the planner supplies local summaries. The source group is the nearest column to the demand:

$$\ell = \mathcal{C}_i^\top q_i, \qquad a^\star = \arg\max_j \ell_j, \qquad c_a = \mathcal{C}_i[:, a^\star].$$

For candidate peer ranges $\mathcal{R}_i^{\text{peer}} = \{R_1, \ldots, R_k\}$, form $C_{\mathcal{R}} = [c_{R_1}, \ldots, c_{R_k}]$. The query residual and all source-to-target directions are

$$u_i = \mathrm{norm}(q_i - c_a),$$
$$V_{a\mathcal{R}} = \mathrm{norm}_{\text{col}}\big(C_{\mathcal{R}} - c_a \mathbf{1}_k^\top\big).$$

The positive alignment vector is

$$\omega_i = [V_{a\mathcal{R}}^\top u_i]_+.$$

Each column of the predictive query matrix is the current demand moved along a candidate group direction, scaled by its alignment:

$$\widetilde{Q}_i = \mathrm{norm}_{\text{col}}\big(q_i \mathbf{1}_k^\top + \eta V_{a\mathcal{R}}\, \mathrm{diag}(\omega_i)\big).$$

Column j is the predicted vector $\widetilde{q}_i(R_j)$. The directional admission vector is

$$\psi_i = \ell_{a^\star}\, \omega_i \odot [\mathrm{diag}(C_{\mathcal{R}}^\top \widetilde{Q}_i)]_+.$$

A candidate range is eligible through prediction when its column has sufficient alignment and similarity. The V1 implementation computes this matrix expression range-by-range in `predict_cache_group_direction`. When a range is selected only through $\widetilde{q}_i(R)$, the pull request marks the query as projected so the peer validates candidates against projected sidecar embeddings rather than the original demand embedding. This is the path used for predicted embedding or cache-tensor pulls.

Figure 3 shows the same field algorithm in two dimensions. The plot is only an illustrative projection of the D_e-dimensional embedding manifold: local field vectors form an ellipsoid with centroid c_a, a peer advertises a candidate ellipsoid with centroid c_R, and the planner moves the current demand q_i along the source-to-peer direction v_{aR}. The shaded lens is the space where the predicted query, local semantic neighborhood, and peer ellipsoid gate agree; those are the entries worth asking the peer to validate and return.

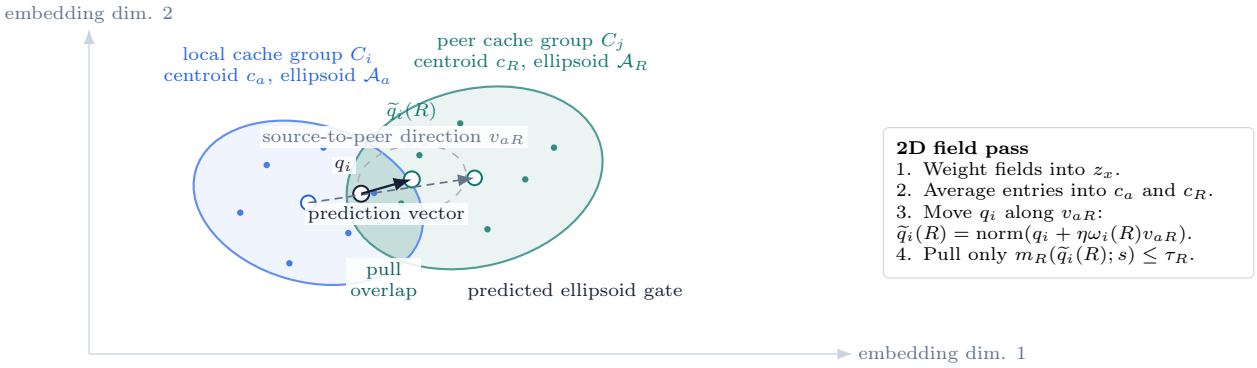

Figure 3: Two-dimensional example of predictive field admission. The real planner runs in the normalized D_e-dimensional embedding space, but the same geometry applies: weighted field vectors define cache-group centroids, low-rank ellipsoids bound advertised ownership, and the directional prediction selects only the peer-cache overlap that is both semantically near the request and inside the peer's published n-dimensional shape.

5.3 Expected-Value Pull Planning

The geometric gate and predictive vector above feed the implementable V1 form of a stronger local optimization problem. Each node should decide which remote ranges to pull by combining three facts: how large the candidate is relative to the KVCache, how much free space remains, and whether the peer range adds a useful direction that the local cache does not already cover.

For node i, let M_i be cache capacity, u_i used bytes, $f_i = (M_i - u_i)_+$ free bytes, and r_i a safety reserve. A pull may also consume an eviction budget E_i, so the byte budget for this planning round is

$$\bar{B}_i = \min\big(B_{\max}, \max(0, f_i - r_i) + E_i\big).$$

Let $X_i = [\sqrt{a_x} z_x]_{x \in C_i}$ be the weighted local embedding matrix, where a_x is access or reuse weight. The local cache covariance is

$$G_i^{\text{cache}} = \lambda I + X_i X_i^\top.$$

A candidate peer range R with centroid c_R, coarse radius ρ_R, default shape $\mathcal{A}_R$, bytes B_R, hotness h_R, and value-class weight W_{v_R} receives two geometric terms. The first is demand alignment; the second is inverse-covariance novelty relative to the local cache, following the same geometry that underlies Mahalanobis-style distance [14]:

$$\begin{aligned}\alpha_i(R) &= \max(0, q_i^\top c_R),\\ \nu_i(R) &= \mathrm{clip}_{[0,1]}\big(\lambda\, c_R^\top (G_i^{\text{cache}})^{-1} c_R\big).\end{aligned}$$

Here q_i is the recent demand centroid. A direction already saturated by local cache entries increases G_i^{cache} and therefore lowers ν_i. A direction that peers cover but the local node lacks has higher inverse-covariance novelty.

Capacity pressure is a separate linear-algebra score multiplier, not just a hard byte limit:

$$\begin{aligned}f_i^+ &= \max(0, f_i - r_i),\\ \varepsilon_i(R) &= \min\big(E_i, \max(0, B_R - f_i^+)\big),\\ P_i(R) &= 1 + \lambda_M B_R / M_i\\ &\quad + \lambda_F B_R / (f_i^+ + \epsilon_0) + \lambda_E \varepsilon_i(R) / M_i.\end{aligned}$$

The B_R/M_i term penalizes values that are large for the whole KVCache. The $B_R/(f_i^+ + \epsilon_0)$ term rises sharply when little post-reserve free space remains. The eviction term charges only the bytes that require displacement and fit inside the current eviction budget E_i.

For the candidate surface visible in peer sketches, form the peer Gram matrix

$$K_{RL} = \big(\max(0, c_R^\top c_L)\big)^2.$$

This matrix detects redundant peer ranges. Pulling two ranges with high K_{RL} wastes memory because they occupy nearly the same semantic direction. The local expected gain for a range is

$$\begin{aligned}\chi_i(R) &= 1 + \lambda_\psi \psi_i(R),\\ g_i(R) &= \frac{W_{v_R}\, \gamma_s(q_i, R)\, \gamma_r(R)}{P_i(R)}\\ &\quad \cdot \alpha_i(R) \nu_i(R) \chi_i(R) \ln(2 + h_R).\end{aligned}$$

The ideal sharing decision is then a byte-constrained quadratic knapsack [16]:

$$\begin{aligned}\max_{x_R \in \{0,1\}} \quad & \sum_R x_R g_i(R) - \lambda_K \sum_{R<L} x_R x_L K_{RL}\\ \text{s.t.} \quad & \sum_R x_R B_R \le \bar{B}_i.\end{aligned}$$

The quadratic term is the linear-algebraic diversity penalty: it prevents the node from spending scarce KV-Cache space on many similar peer ranges. A practical

online planner uses lazy greedy marginal gain:

$$\Delta_i(R \mid S) = g_i(R) - \lambda_K \sum_{L \in S} K_{RL},$$
$$R^\star = \arg\max_R \Delta_i(R \mid S)/B_R.$$

It repeatedly adds $R^\star$ while $\Delta_i(R^\star \mid S) > 0$ and the byte budget remains. This gives Pluribus a strong local algorithm: metadata sharing exposes candidate subspaces, while value movement is reserved for ranges that are aligned with demand, novel relative to the local cache, diverse relative to other peer candidates, and affordable under current KVCache pressure.

At read time, a node first checks its local cache. If no acceptable local hit exists, it forms a demand signal $d = (model, embedding, hotness)$, projects it into the sketch space, and scores peer ranges. A range is eligible when the projected demand passes the coarse radius and, when present, the ellipsoid score; group-direction prediction $\widetilde{q}_i(R)$ uses the same gate with the projected query. The utility score balances semantic confidence, radius penalty, density, value-class priority, expected byte cost, hotness, TTL, peer quality, and directional group confidence.

The selected peer receives a bounded range pull request. The response contains candidate items, not an unbounded range dump. The receiver validates every item and inserts accepted values through the same semantic write path as local writes. Future reads are then local.

This design intentionally separates control and data. Blossom moves small metadata. shardcache moves values. Pluribus decides when a value is worth moving.

Viewed as a P2P network, every cache node is both a publisher and a consumer, following the symmetry common in peer-to-peer lookup systems [21]. Nodes gossip bounded sketches through Blossom, retain full values locally, and issue direct shardcache pulls only to peers selected by the local planner. There is no central cache coordinator and no broadcast replication of values; only small metadata uses the epidemic-style dissemination pattern [23, 6].

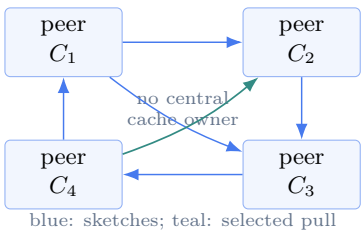

Figure 4: P2P network view. Each peer advertises compact ownership sketches and keeps values local. A miss is planned locally, then the node pulls a bounded range directly from the selected peer. Values are never broadcast through a central cache service.

6 Projections

The shared-repository case improves because request diversity is high at the prompt surface but low in embedding space. Let q_t be the embedding of a query from any LLM instance at time t, and let C_i be the local cache on node i. If many workers operate on the same repository, the probability that a useful value exists in some peer cache is high:

$$P(\exists j \neq i, x \in C_j : \mathrm{sim}(q_t, x) \geq \theta)$$

increases with the number of workers and recent codebase reuse. The useful question is not whether the cluster has a nearby value, but whether the local node can discover and pull it within byte and latency budgets.

For a single shared repository, embeddings cluster around one project manifold. The planner should therefore see fewer unrelated peer ranges, higher semantic confidence, and fewer wasted pulls. For many unrelated repositories sharing the same fixed cache capacity, sketches become wider, metadata grows, and useful candidate density falls. This density argument uses the same occupancy intuition as balls-and-bins and Poissonized cache models [17]. The expected behavior is:

$$\mathrm{Hit}_{\mathrm{shared}} \gg \mathrm{Hit}_{\mathrm{many}}, \qquad \mathrm{Miss}_{\mathrm{shared}} \ll \mathrm{Miss}_{\mathrm{many}}.$$

Exact-file reuse is stricter. It depends on repeated demand for the same file or exact cache key. Subsystem reuse is the more natural semantic target for a coding assistant because nearby files, summaries, test results, and interface fragments may all be useful context for the same task, just as retrieval-based systems treat related passages as useful generation context [10].

6.1 Implemented Scaling Envelope

The pull planner in Section 5.3 gives the full local optimization objective. Figures 6 and 7 plot the implemented scalarized envelope used by `project_sharing_policy`, with repository breadth, repository size, and embedding variance made explicit. The scalarization keeps the same factors as the quadratic policy: geometric eligibility, peer availability, byte admission, free-space pressure, novelty, and diversity.

Let θ be the semantic acceptance score, $r_t + s$ the effective range gate, and $\widehat{c}_i$ the expected nearest useful peer cosine for node i. The two geometric gates are

$$S_\theta(i) = \sigma_\theta(\widehat{c}_i - \theta),$$
$$G_r(i) = \sigma_r\big((r_t + s) - (1 - \widehat{c}_i)\big),$$

where

$$\sigma_\tau(x) = \frac{1}{1 + \exp(-x/\tau)}.$$

The logistic gate is a smooth surrogate for a hard threshold, a standard device in probabilistic scoring models [15]. The implemented geometry term is their product:

$$\Gamma_i = S_\theta(i) G_r(i).$$

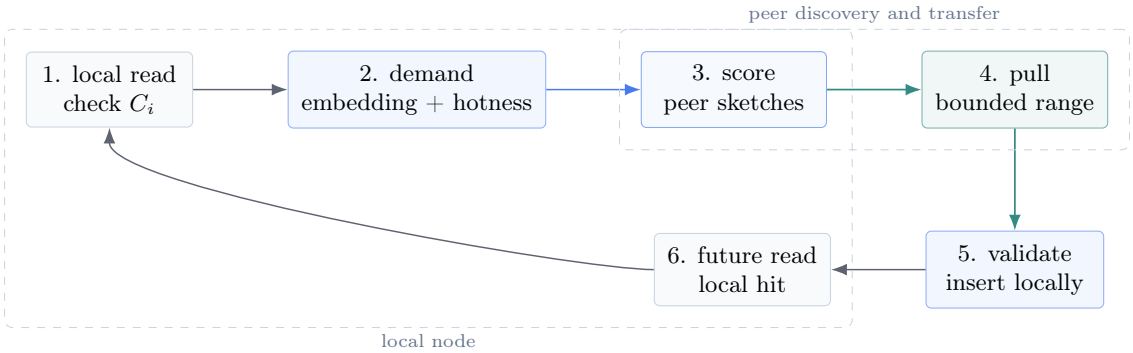

Figure 5: Remote value pull lifecycle. The left side stays on the local node. A miss becomes a demand signal, peer sketches are scored, one bounded range is pulled, validated, and inserted. The next similar request is a local hit.

It is close to one only when the peer range is expected to pass final semantic validation and the projected demand lies inside the advertised metadata range.

Let N_i be effective visible peer entries for the shared project, H_i the subsystem working-set surface, I_{req} the requested item count, B_v the value size, and $\bar{B}_i$ the round byte budget from Section 5.3. The availability and byte-admission gates are

$$A_i = 1 - \exp(-N_i/H_i), \qquad L_i = 1 - \exp(-\bar{B}_i/(I_{\text{req}} B_v)).$$

Now expose the global scale terms hidden inside N_i. Let M_g be global cache bytes visible to the cluster, R_{repo} be the number of active repositories sharing that cache, S_{repo} be repository working-set size relative to the baseline shared-repo benchmark, B_v be mean value bytes, σ_ℓ^2 be within-repository embedding variance, σ_g^2 be global embedding variance, and d_{eff} be the intrinsic dimension of the useful project manifold [18]. The embedding-space and working-set dilution is

$$\Omega_g = R_{\text{repo}} S_{\text{repo}} \left(\frac{\sigma_g^2}{\sigma_\ell^2} \right)^{d_{\text{eff}}/2}.$$

If ζ is the fraction of global cache entries visible, fresh, and eligible for the queried project, the useful peer-entry count is

$$N_i^{\text{eff}} = \frac{\zeta M_g}{B_v \Omega_g}.$$

The apparent 1:1 cache law is therefore a density law. Holding geometry, metadata, and byte gates constant,

$$A_i = 1 - \exp\left(-\frac{\zeta M_g}{B_v H_i \Omega_g} \right).$$

In the low-density regime $N_i^{\text{eff}} \ll H_i$ and $\kappa A_i \Xi_i \ll 1$, where Ξ_i is the admitted value surface defined below, this gives

$$\widehat{H}_i \approx \kappa \Xi_i \frac{\zeta M_g}{B_v H_i \Omega_g}.$$

Thus global cache space scales 1:1 with useful embedding volume only while Ξ_i is approximately constant. Doubling cache bytes offsets doubling repository breadth, repository working-set size, or embedding-volume dilution, but it does not offset a geometry gate that has already rejected the peer range, a metadata cap that no longer exposes the useful ranges, or local capacity pressure that prevents admission.

Density-law proof sketch. Let $y = \zeta M_g/(B_v H_i \Omega_g)$. The availability gate is $A_i = 1 - e^{-y}$, so $A_i \approx y$ in the low-density regime. If Ξ_i and κ are fixed and the target hit rate is $\widehat{H}_i = h < 1$, then $-\ln(1-h) = \kappa \Xi_i A_i \approx \kappa \Xi_i \zeta M_g/(B_v H_i \Omega_g)$. Solving for M_g gives

$$M_g \approx \frac{-\ln(1-h)\, B_v H_i}{\kappa \Xi_i \zeta}\, \Omega_g.$$

The coefficient is constant only while geometry, metadata visibility, novelty, diversity, and byte admission remain stable; outside that regime the measured curves may bend or saturate.

The finite metadata sketch adds one more implemented visibility term. After occupancy-aware range construction, the renderer does not impose a visibility loss through the validated 16-repository point. Beyond that point, repository breadth can exceed the 16 KiB app-state surface and fewer useful summaries are visible [6]. Define the post-threshold decay

$$g_\phi(R) = \frac{1}{1 + ((R - R_0)/\tau_\phi)^{p_\phi}}.$$

The current bounded scalar is

$$\phi_i^{\text{sketch}}(R_{\text{repo}}) = \begin{cases} 1, & R_{\text{repo}} \le R_0, \\ \max\big(\phi_{\min}, g_\phi(R_{\text{repo}})\big), & R_{\text{repo}} > R_0, \end{cases}$$

with $R_0 = 16$, $\phi_{\min} = 0.18$, $\tau_\phi = 96$, and $p_\phi = 1.35$ in the figure renderer. This term is monotone after R_0; any residual nonmonotone measurement in the benchmark comes from the measured range construction and metadata bytes, not from projection compensation. The smooth 1:1 density law still governs N_i^{eff}, while ϕ_i^{sketch} captures the finite-sketch visibility limit actually used by the implementation.

Capacity pressure is exactly the code-level multiplier applied before a range can create value:

$$\begin{aligned} f_i^+ &= \max(0, f_i - r_i), \\ \varepsilon_i(R) &= \min\big(E_i, \max(0, B_R - f_i^+)\big), \\ P_i(R) &= 1 + \lambda_M B_R / M_i \\ &\quad + \lambda_F B_R / (f_i^+ + \epsilon_0) + \lambda_E \varepsilon_i(R) / M_i. \end{aligned}$$

Here B_R is the candidate byte count, M_i is local KVCache capacity, f_i is free space, r_i is reserve space, and E_i is eviction budget. The first term charges whole-cache footprint, the second charges local scarcity, and the third charges displacement.

The V1 implementation carries inverse-covariance novelty and peer-Gram diversity as bounded factors $\bar{\nu}_i, \bar{\mathcal{D}}_i$. In the full matrix policy, novelty is the Mahalanobis-style term $c_R^\top (G_i^{\text{cache}})^{-1} c_R$ [14], while diversity comes from the off-diagonal peer Gram matrix used by Pluribus; in the current planner they are configurable scalars because range sketches do not yet publish enough local covariance to reconstruct those matrices. The admitted value surface is

$$\Xi_i = \frac{\Gamma_i L_i \bar{\nu}_i \bar{\mathcal{D}}_i \phi_i^{\text{sketch}}}{P_i(R)}.$$

The projected subsystem hit envelope and metadata selectivity are

$$\begin{aligned} \widehat{H}_i &= 1 - \exp(-\kappa A_i \Xi_i), \\ \widehat{M}_i &= G_r(i) \bar{\mathcal{D}}_i \phi_i^{\text{sketch}}. \end{aligned}$$

Finally, local reuse reduces remote movement. With N_i^{local} local entries, local reuse surface H_i^{local}, and cap c_{local},

$$\begin{aligned} A_i^{\text{local}} &= \min\big(1 - \exp(-N_i^{\text{local}} / H_i^{\text{local}}), c_{\text{local}}\big), \\ \widehat{P}_i^{\text{pull}} &= \widehat{H}_i (1 - A_i^{\text{local}}). \end{aligned}$$

The scale law has four important limits. First, A_i saturates, so extra global cache has diminishing returns once the local project surface is covered. Second, $\widehat{c}_i$ falls as global embedding variance grows; once S_θ or G_r collapses, extra cache cannot recover the miss without a better embedding or a wider safe range. Third, the 16 KiB Blossom cap bounds ζ and ϕ_i^{sketch}; entries that are not advertised do not contribute to N_i^{eff}, and advertised ranges that are too broad do not contribute much value. Fourth, $P_i(R)$ and L_i enforce local byte admission, so global abundance does not imply local capacity.

Figure 6 uses the same symbols. Panel A shows the ideal cache-scaling curve for fixed Ω_g, with $\phi_i^{\text{sketch}} = 1$. Panel B varies repository breadth with the finite-sketch visibility term enabled and overlays a measured benchmark row at every plotted x value. Panel C varies S_{repo} and overlays the matching measured monorepo-size grid, showing that a single very large monorepo can dilute cache density even when $R_{\text{repo}} = 1$. Panel D varies σ_g^2/σ_ℓ^2 to show where embedding variance turns the problem from cache-limited into geometry-limited. Figure 7 solves the low-density formula for the cache multiple needed to hold a target hit rate, exposing the 1:1 slope between M_g and Ω_g before saturation, metadata, and geometry gates dominate.

7 Benchmarks

7.1 Method

The shared-repo benchmark lives in the Pluribus simulator crate and is generated by the matching shell runner. The runner performs formatting, clippy with warnings denied, and a release benchmark run. Raw output is stored in the repository's benchmark results directory.

All scenarios use 64 physical cache nodes, 4,096 logical LLM instances, 6,144 seeded cache entries, and 2 KiB stored benchmark values. Sketch payloads are bounded by the 16 KiB Blossom application-state cap [6]. The Figure 6 coverage grid uses 256-dimensional synthetic embeddings and runs three 2,500-query repeats at every repository-breadth and repository-size x value. The breadth grid keeps $S_{\text{repo}} = 1$ while varying R_{repo}. The size grid keeps $R_{\text{repo}} = 1$ while varying S_{repo}.

7.2 Results

The compact shared-repo case is the favorable Pluribus path. Every query is answered locally or repaired by a bounded remote pull at the subsystem level. Exact-file hit rate stays near 37%, which is lower because semantic reuse and exact object reuse measure different things.

The occupancy-aware sketch pass removes the earlier repository-breadth cliff. With $S_{\text{repo}} = 1$, subsystem hit rate stays above 93% through 16 repositories, including 96.03% at the formerly weak 16-repository point. The curve then falls to 51.47% at 32 repositories, 40.53% at 64, and 25.63% at 128 as per-node repository coverage is diluted. The monorepo-size grid shows the complementary working-set effect: with $R_{\text{repo}} = 1$, subsystem hit rate remains 99.65% at 2×, 94.68% at 4×, 83.24% at 8×, and remains in the 57–65% band at 16× and 32×. The 32× row should not be read as a monotonic scaling law: it publishes about 456 KiB of aggregate sketch metadata, versus about 144 KiB at 16×, so extra visible ranges partially offset the larger embedding surface in this deterministic harness. These results validate the refined hypothesis: shared coding repositories are a strong fit when many LLM workers reuse one compact semantic working set; very large monorepos and many unrelated repositories both require proportionally more cache and metadata surface.

7.3 Tuning Sweep

The tuning sweep reuses the shared-repository workload with one repeat per point and 2,500 queries. It varies score threshold, effective range gate, pull budget, and free

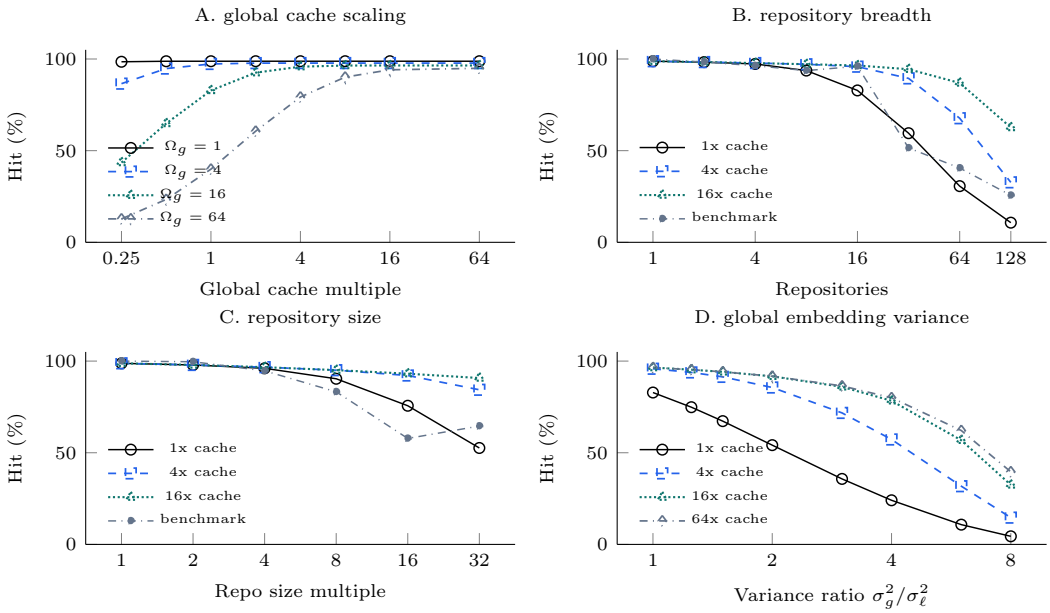

Figure 6: Scale-law projection for shared-cache decay. Panel A varies global cache size while holding the effective embedding dilution Ω_g fixed, with finite-sketch visibility disabled to show the clean density law. Panel B enables the 16 KiB sketch-visibility term and overlays the connected three-repeat, 256-dimensional measured repository-breadth benchmark. Panel C overlays the matching repository-size benchmark. Panel D varies global embedding variance at 16 repositories, showing where score-gate loss makes extra cache less effective.

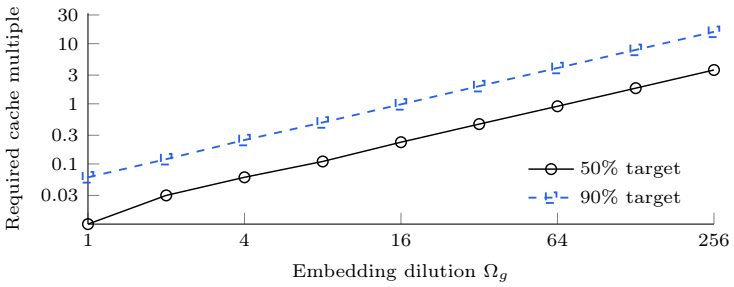

Figure 7: Cache multiple required to hold a target subsystem-hit envelope as the effective embedding dilution Ω_g grows. The near-linear slope is the density-limited regime of the proof; departures occur when saturation, metadata selectivity, or geometry gates dominate.

space. The score and range sweeps use the compressed benchmark payloads above. The capacity sweep models logical KV-cache movement with 2 MiB tensor chunks, 8-item repairs, a 64 MiB default round budget, and a 32 GiB local cache. The measured subsystem hit rate stays at 100% across the tested threshold range because the shared-project manifold is tight enough that each miss can still be repaired by a small remote pull. The pull-pressure sweep is more sensitive: tighter byte and free-space budgets reduce the model's expected remote movement, while the measured implementation remains conservative because it attempts only the highest-ranked bounded pulls. This is consistent with bounded online selection rather than offline-optimal cache replacement [19, 16].

7.4 Existing Validation

The broader workspace still validates Pluribus under blind-cache and metadata-mode workloads. The all-target test suite covers corrupt responses, duplicate items, stale TTLs, wrong model ids, wrong dimensions, partitions, offline nodes, deterministic replay, blind observed writes, and learned traffic transitions. The real-workload benchmark covers RAG chunks, conversation memory, tool results, prompt fragments, and mixed LLM cache values [10]. The shared-repo benchmark adds the concentrated coding-agent case.

8 Future Additions

The next version should connect the shared-repo benchmark to a server-mode deployment. The current evidence

Table 1: Figure 6 benchmark coverage grid. Hit rates are semantic unless noted as exact-file.

Scenario	Repos	Size	Dim	Cosine	Repo	Subsys	Exact	Miss	Top-1 subsys
breadth-1	1	1.0	256	0.966	100.00%	100.00%	35.59%	0.00%	100.00%
breadth-2	2	1.0	256	0.966	98.32%	98.32%	30.19%	1.68%	96.27%
breadth-4	4	1.0	256	0.966	96.35%	96.35%	27.67%	3.65%	99.11%
breadth-8	8	1.0	256	0.966	93.67%	93.67%	27.31%	6.33%	99.78%
breadth-16	16	1.0	256	0.966	96.03%	96.03%	26.11%	3.97%	100.00%
breadth-32	32	1.0	256	0.966	51.47%	51.47%	12.71%	48.53%	99.60%
breadth-64	64	1.0	256	0.966	40.53%	40.53%	10.04%	59.47%	100.00%
breadth-128	128	1.0	256	0.966	25.63%	25.63%	6.40%	74.37%	100.00%
size-1x	1	1.0	256	0.966	100.00%	100.00%	38.04%	0.00%	100.00%
size-2x	1	2.0	256	0.960	99.65%	99.65%	29.84%	0.35%	100.00%
size-4x	1	4.0	256	0.954	94.68%	94.68%	25.49%	5.32%	100.00%
size-8x	1	8.0	256	0.945	83.24%	83.24%	21.89%	16.76%	100.00%
size-16x	1	16.0	256	0.943	57.81%	57.81%	14.11%	42.19%	100.00%
size-32x	1	32.0	256	0.932	64.55%	64.55%	16.89%	35.45%	100.00%

Table 2: Planner and metadata costs. Metadata bytes are aggregate sketch bytes per refresh across the cache cluster.

Scenario	Planned pulls	Attempts	Pulled MiB	Metadata bytes	P50 us	P95 us	Wall ms
breadth-1	3,010.7	403.0	1.934	86,978	16.3	7,819.0	14,684.3
breadth-2	7,821.3	1,234.7	3.605	143,020	18.3	7,873.3	17,451.3
breadth-4	12,944.0	2,087.7	4.444	249,642	4,045.7	6,906.3	19,719.0
breadth-8	15,853.3	2,452.7	4.747	573,093	4,378.3	6,573.3	22,408.0
breadth-16	17,858.7	2,629.0	4.740	861,438	4,679.0	6,696.3	45,866.0
breadth-32	19,096.0	5,381.3	2.400	780,889	3,932.3	5,871.0	25,450.7
breadth-64	19,464.0	5,935.3	1.868	898,428	3,823.7	6,066.3	38,339.0
breadth-128	19,709.3	6,574.0	1.185	867,435	3,570.3	5,734.7	59,592.0
size-1x	3,253.3	414.7	2.299	101,423	16.3	4,758.3	7,784.7
size-2x	8,304.0	1,224.0	3.610	82,219	18.3	3,997.7	9,406.7
size-4x	12,706.7	2,458.0	3.909	74,210	2,020.7	3,853.7	10,545.3
size-8x	16,170.7	3,402.3	3.754	157,061	2,127.3	3,147.3	10,507.3
size-16x	17,992.0	4,962.7	2.546	143,847	1,455.0	2,706.0	10,084.0
size-32x	19,010.7	4,586.7	3.066	455,742	2,573.7	3,334.0	14,811.3

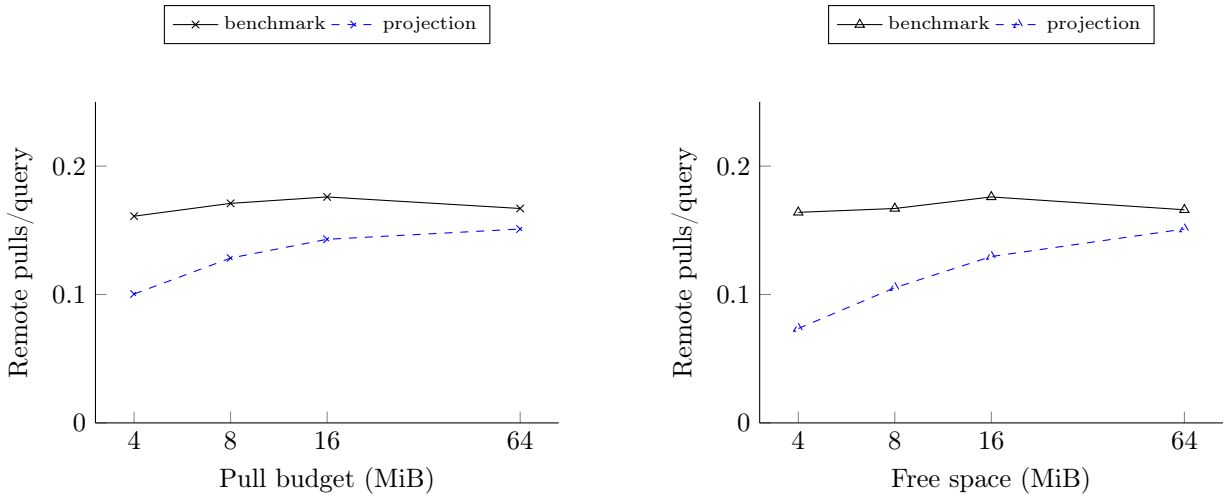

Figure 8: Capacity-pressure sweep for the same workload. The pull-budget plot constrains how many bytes a remote repair may return. The free-space plot applies the algebraic free-space budget as the maximum value movement available in the planning round. The projection tracks the expected pull pressure from the capacity multiplier $P_i(R)$.

is algorithmic and in process. A production study should measure shardcache request latency, serialization overhead, network transfer, process scheduling, and memory pressure while many real LLM workers share one project cache [2, 3].

Eviction policy is also important. A shared repository can make many values look useful, but finite memory still requires choosing between exact-file objects, subsystem summaries, RAG chunks, test outputs, and prompt fragments. Pluribus should feed remote-hit outcomes back into value-class and byte-budget policy so repeated subsystem wins increase priority while unused pulls decay, building on the classic tension between optimal replacement and practical online cache policies [19, 20].

Governance should become first-class in the semantic path. Coding agents may serve different users, branches,

access scopes, or policy versions over the same repository. Pluribus already carries governance bytes in semantic entries; future work should standardize how applications enforce branch, tenant, and authorization boundaries before serving semantic hits. This matters because semantic similarity alone is not an access-control decision; authorization must remain an application policy boundary [24].

Finally, the blind mode and semantic mode should converge in reporting. A single observer view should explain whether a pull was motivated by exact key, semantic range, blind field prediction, peer quality, or fallback policy.

9 Conclusion

Pluribus improves distributed cache behavior by moving only the values that a node has reason to reuse. Blossom carries bounded sketches, shardcache carries validated values, and each node keeps prediction local [6]. The design works in a metadata-blind baseline and becomes stronger when an application can safely provide embeddings and governance metadata.

The shared coding repository benchmark is the clearest positive case so far. With 4,096 logical LLM instances over one project, embeddings cluster tightly enough that the cache reaches 100.00% subsystem-level semantic hits and 100.00% top-1 subsystem prediction on the non-local path. The expanded coverage grid shows the two main decay paths: an 8x monorepo reaches 83.24% subsystem hit rate, a 32x monorepo reaches 64.55%, and a normalized 128-repository breadth case reaches 25.63% once finite sketch visibility and metadata saturation dominate. The occupancy-aware sketch builder also removes the former 16-repository cliff, raising that breadth point to 96.03%. That contrast is useful product guidance: Pluribus should be deployed first where many LLM workers share one compact semantic working set, and larger monorepos should be sized by their working-set surface, not merely counted as one repository.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, *Attention Is All You Need*, Advances in Neural Information Processing Systems, 2017. Available: https://arxiv.org/abs/1706.03762.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica, *Efficient Memory Management for Large Language Model Serving with PagedAttention*, 2023. Available: https://arxiv.org/abs/2309.06180.

[3] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang, *LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference*, 2025. Available: https://arxiv.org/abs/2510.09665.

[4] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen, *KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider*, 2025. Available: https://arxiv.org/abs/2506.02634.

[5] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu, *Mooncake: Trading More Storage for Less Computation – A KVCache-centric Architecture for Serving LLM Chatbot*, 23rd USENIX Conference on File and Storage Technologies, 2025. Available: https://www.usenix.org/system/files/fast25-qin.pdf.

[6] Devon Tietjen, *Blossom Consensus Protocol v2.0.0-pre-release*, 2026. Available: https://github.com/eroica-dev/blossom/blob/main/Blossom-Consensus-Protocol-v2-Whitepaper.pdf.

[7] Jiaxing Li, Chi Xu, Feng Wang, Isaac M. von Riedemann, Cong Zhang, and Jiangchuan Liu, *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*, 2024. Available: https://arxiv.org/abs/2406.00025.

[8] Sajal Regmi and Chetan Phakami Pun, *GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching*, 2024. Available: https://arxiv.org/abs/2411.05276.

[9] Dvir David Biton and Roy Friedman, *From Exact Hits to Close Enough: Semantic Caching for LLM Embeddings*, 2026. Available: https://arxiv.org/abs/2603.03301.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. Available: https://arxiv.org/abs/2005.11401.

[11] Gerard Salton, Anita Wong, and C. S. Yang, *A Vector Space Model for Automatic Indexing*, Communications of the ACM, 1975.

[12] Jeff Johnson, Matthijs Douze, and Herve Jegou, *Billion-scale similarity search with GPUs*, IEEE Transactions on Big Data, 2017. Available: https://arxiv.org/abs/1702.08734.

[13] Karl Pearson, *On Lines and Planes of Closest Fit to Systems of Points in Space*, Philosophical Magazine, 1901.

[14] Prasanta Chandra Mahalanobis, *On the Generalised Distance in Statistics*, Proceedings of the National Institute of Science of India, 1936.

[15] Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[16] Hans Kellerer, Ulrich Pferschy, and David Pisinger, *Knapsack Problems*, Springer, 2004.

[17] Michael Mitzenmacher and Eli Upfal, *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*, Cambridge University Press, 2005.

[18] Elizaveta Levina and Peter J. Bickel, *Maximum Likelihood Estimation of Intrinsic Dimension*, Advances in Neural Information Processing Systems, 2005.

[19] Laszlo A. Belady, *A Study of Replacement Algorithms for a Virtual-Storage Computer*, IBM Systems Journal, 1966.

[20] Nimrod Megiddo and Dharmendra S. Modha, *ARC: A Self-Tuning, Low Overhead Replacement Cache*, 2nd USENIX Conference on File and Storage Technologies, 2003.

[21] Petar Maymounkov and David Mazieres, *Kademlia: A Peer-to-peer Information System Based on the XOR Metric*, International Workshop on Peer-to-Peer Systems, 2002. Available: https://pdos.csail.mit.edu/~petar/papers/maymounkov-kademlia-lncs.pdf.

[22] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels, *Dynamo: Amazon's Highly Available Key-value Store*, ACM Symposium on Operating Systems Principles, 2007.

[23] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry, *Epidemic Algorithms for Replicated Database Maintenance*, ACM Symposium on Principles of Distributed Computing, 1987.

[24] Jerome H. Saltzer and Michael D. Schroeder, *The Protection of Information in Computer Systems*, Proceedings of the IEEE, 1975.

[25] Leslie Lamport, Robert Shostak, and Marshall Pease, *The Byzantine Generals Problem*, ACM Transactions on Programming Languages and Systems, 1982.

10 Variable Glossary

This glossary collects the symbols used in the protocol, projection model, and benchmark analysis.

Indices, Sets, and Spaces

i, j
: Cache-node indices. Node i is usually the local planning node, while $j \neq i$ denotes another peer.

R, L
: Advertised cache ranges. R is the candidate range being scored; L is usually another selected or compared range.

x, t
: Cache-entry and time indices. x identifies one cached value or sidecar; t indexes a query event.

$C_i, \mathcal{C}_i$
: C_i is node i's local cache contents. $\mathcal{C}_i$ is a matrix of visible group centroids; the V1 implementation uses advertised peer centroids for directional prediction.

$\mathcal{R}_i^{\text{peer}}, S$
: $\mathcal{R}_i^{\text{peer}}$ is the candidate peer-range set visible to node i. S is the set of ranges already selected by the greedy planner.

D_e, p, k, m_i
: D_e is embedding dimension, p is the number of embedding fields per structured entry, k is the number of candidate peer ranges in a planning matrix, and m_i is the number of ranges in node i's published sketch.

$\mathbf{1}$
: An all-ones vector with dimension implied by the expression.

Sketches, Embeddings, and Ranges

Σ_i
: The bounded Blossom application-state sketch published by node i.

e_x, e_q, z_x, q_i, q_t
: e_x and e_q are raw entry and query embeddings. $z_x = \pi(e_x)$ is the normalized projected entry embedding. $q_i = \pi(e_q)$ is node i's current demand vector. q_t is a query embedding at time t.

$\pi, d(a, b), \text{sim}$
: π is the projection and normalization function. $d(a, b) = 1 - a^\top b$ is cosine distance for normalized vectors. sim is semantic similarity.

$c_R, \bar{c}_R, \rho_R, \mathcal{A}_R$
: c_R is the normalized centroid advertised for range R. $\bar{c}_R$ is the unnormalized weighted centroid before normalization. ρ_R is the coarse observed cosine-distance range radius. $\mathcal{A}_R$ is the default low-rank ellipsoid descriptor published for range admission.

$U_R, u_{R\ell}, k_R, a_{R\ell}, \sigma_{\perp,R}, P_R^\perp, \delta_s, r_{R\ell}, r_{\perp,R}, \tau_R, m_R$
: U_R stores k_R advertised principal axes, $u_{R\ell}$ is one axis, $a_{R\ell}$ is its p95 member radius, $\sigma_{\perp,R}$ is the p95 residual radius over omitted dimensions, $P_R^\perp$ projects onto the omitted subspace, δ_s converts cosine-distance slack to Euclidean radius slack, $r_{R\ell}$ and $r_{\perp,R}$ are slack-expanded radii, τ_R is the fitted ellipsoid threshold, and m_R is the resulting Mahalanobis-style admission score.

N_R, n_R, b_R, B_R
: N_R, b_R are raw entry count and raw byte count. n_R, B_R are the planner-capped count and byte values.

$h_R, h_{q_i}, v_R, W_{v_R}, \mu_{v_R}$
: h_R is range hotness and h_{q_i} is demand hotness. v_R is value class, W_{v_R} is its utility weight, and μ_{v_R} is its byte-cost multiplier.

$r_{\min}, r_t, r_{\max}, s, \theta$
: $r_{\min}$ is the weak-compactness floor, r_t is the publication target radius, $r_{\max}$ is the merge radius limit, s is pull-time planner slack, and θ is the semantic acceptance threshold.

$I_{\min}, I_{\max}, B_{\max}, n_\ell, N_{\text{med}}, Q_{0.95}$
: $I_{\min}$ is the configured minimum entries per published range, $I_{\max}$ caps planner range entry count, $B_{\max}$ caps range bytes and pull budget, n_ℓ is the implemented split leaf floor, N_{med} is the medium-occupancy split bound, and $Q_{0.95}$ is the p95 member-distance quantile.

Linear Algebra and Prediction

a_x, w_x, w_R, β
: a_x is access or reuse weight. w_x is publication hotness weight. w_R is the vector of range-entry weights. β is the vector of field-importance weights.

$F_x, Z_R, \mathcal{E}_R$
: F_x is the matrix of per-field embeddings for entry x. Z_R stacks entry vectors for range R. $\mathcal{E}_R$ is the tensor form of all field embeddings in a range.

X_i, G_i^{cache}
: X_i is the weighted local embedding matrix. G_i^{cache} is the regularized local cache covariance used for novelty.

$\alpha_i(R), \nu_i(R)$
: $\alpha_i(R)$ is demand alignment with range R. $\nu_i(R)$ is inverse-covariance novelty relative to the local cache.

$\ell, a^\star, c_a$
: $\ell = \mathcal{C}_i^\top q_i$ is the source-group similarity vector. $a^\star$ is the nearest visible group index and c_a is its centroid.

$C_{\mathcal{R}}, u_i, V_{a\mathcal{R}}, v_{aR}$
: $C_{\mathcal{R}}$ stacks candidate peer centroids. u_i is the normalized query residual from the nearest source group. $V_{a\mathcal{R}}$ stacks source-to-target direction vectors. v_{aR} is the column direction from the source group a to candidate range R.

$\omega_i, \eta, \widetilde{Q}_i, \widetilde{q}_i(R)$
: ω_i is the positive alignment vector, η scales directional prediction, $\widetilde{Q}_i$ is the predictive query matrix, and $\widetilde{q}_i(R)$ is the predicted vector for range R.

$\psi_i(R), \chi_i(R)$
: $\psi_i(R)$ is directional admission confidence. $\chi_i(R)$ is the directional gain multiplier derived from $\psi_i(R)$.

$K_{RL}, x_R, \Delta_i(R \mid S), R^\star$
: K_{RL} is the peer Gram similarity penalty between ranges. x_R is a binary pull decision. $\Delta_i(R \mid S)$ is greedy marginal gain, and $R^\star$ is the next selected range.

Capacity and Utility

$M_i, u_i, f_i, f_i^+, r_i, E_i, \bar{B}_i$
: M_i is local KVCache capacity, u_i is used bytes, f_i is free space, $f_i^+ = \max(0, f_i - r_i)$ is post-reserve free space, r_i is reserve space, E_i is eviction budget, and $\bar{B}_i$ is the planning-round byte budget.

$\varepsilon_i(R), P_i(R), \epsilon_0$
: $\varepsilon_i(R)$ is eviction-budget-capped displacement pressure for range R. $P_i(R)$ is the capacity-pressure multiplier. ϵ_0 prevents division by zero.

$\gamma_\rho(q_i, R), \gamma_{\mathcal{A}}(q_i, R), \gamma_s(q_i, R), \gamma_r(R)$
: γ_ρ is coarse-radius confidence, $\gamma_{\mathcal{A}}$ is ellipsoid-shape confidence, γ_s is their implemented membership confidence for demand q_i, and γ_r penalizes wide advertised ranges.

$D_R, \Lambda_R^{\text{byte}}, \Phi_i(q_i, R), U_i(q_i, R)$
: D_R is the density multiplier, Λ_R^{byte} is the byte-cost multiplier, Φ_i is the implemented range gain, and U_i is planner utility.

$g_i(R), \bar{\nu}_i, \bar{\mathcal{D}}_i$
: $g_i(R)$ is full local expected gain. $\bar{\nu}_i$ and $\bar{\mathcal{D}}_i$ are scalarized novelty and diversity factors used by the V1 planner.

$\lambda, \lambda_r, \lambda_d, \lambda_b, \lambda_M, \lambda_F, \lambda_E, \lambda_\psi, \lambda_K, \kappa, U_{\min}$
: Regularization and tuning constants. $U_{\min}$ is the minimum utility needed to emit a pull.

Projection and Benchmark Envelope

$\widehat{c}_i, \sigma_\tau, S_\theta(i), G_r(i), \Gamma_i$
: $\widehat{c}_i$ is expected nearest useful peer cosine. σ_τ is a logistic gate. S_θ is the score gate, G_r is the range gate, and Γ_i is their geometry product.

$N_i, H_i, I_{\text{req}}, B_v, A_i, L_i$
: N_i is visible peer entries, H_i is hot working-set surface, I_{req} is requested item count, B_v is value size, A_i is availability, and L_i is byte admission.

$\Xi_i, \widehat{H}_i, \widehat{M}_i, \widehat{P}_i^{\text{pull}}$
: Ξ_i is admitted value surface, $\widehat{H}_i$ is projected subsystem hit envelope, $\widehat{M}_i$ is metadata selectivity, and $\widehat{P}_i^{\text{pull}}$ is remote-pull pressure.

$M_g, R_{\text{repo}}, S_{\text{repo}}, \sigma_\ell^2, \sigma_g^2, d_{\text{eff}}$
: M_g is global cache bytes, R_{repo} is active repository breadth, S_{repo} is repository working-set size relative to the baseline shared-repo benchmark, σ_ℓ^2 is within-repository embedding variance, σ_g^2 is global embedding variance, and d_{eff} is the intrinsic dimension of the useful manifold.

$\Omega_g, \zeta, N_i^{\text{eff}}$
: Ω_g is repository-count, repository-size, and embedding-volume dilution, ζ is the eligible visible-cache fraction, and N_i^{eff} is the useful peer-entry count after dilution.

ϕ_i^{sketch}
: ϕ_i^{sketch} is the finite app-state sketch-visibility factor.

$g_\phi, R_0, \phi_{\min}, \tau_\phi, p_\phi$
: Scalar constants for the implemented sketch-visibility envelope: the post-threshold decay function, no-penalty breadth cutoff, visibility floor, breadth-decay scale, and breadth-decay exponent.

$N_i^{\text{local}}, H_i^{\text{local}}, c_{\text{local}}, A_i^{\text{local}}$
: Local reuse inputs: local entries, local working-set surface, local reuse cap, and resulting local availability.

$\text{Hit}_{\text{shared}}, \text{Hit}_{\text{many}}, \text{Miss}_{\text{shared}}, \text{Miss}_{\text{many}}$
: Projected hit and miss rates for the shared-repository and many-repository benchmark regimes.